EXPERIMENT - 7

AIM

To identify functional dependencies in a denormalized Sales table and normalize it step-by-step from UNF to 3NF and BCNF, then verify lossless join and dependency preservation using SQL queries.

OBJECTIVES

- To identify functional dependencies and candidate keys in a denormalized Sales table.
- To represent the original data in Unnormalized Form (UNF) and convert it to First Normal Form (1NF).
- To decompose the relation into 2NF, 3NF, and Boyce–Codd Normal Form (BCNF).
- To verify that the decompositions are lossless and preserve the required functional dependencies.
- To use SQL queries to create the normalized tables and inspect the resulting data.

DATABASE USED

The experiment uses a MySQL database named CompanyDB and a sample Sales dataset created specifically to demonstrate normalization.

```
CREATE DATABASE IF NOT EXISTS CompanyDB;
USE CompanyDB;
```

SOURCE CODE / SQL QUERIES

1. CREATE THE DENORMALIZED SALES TABLE

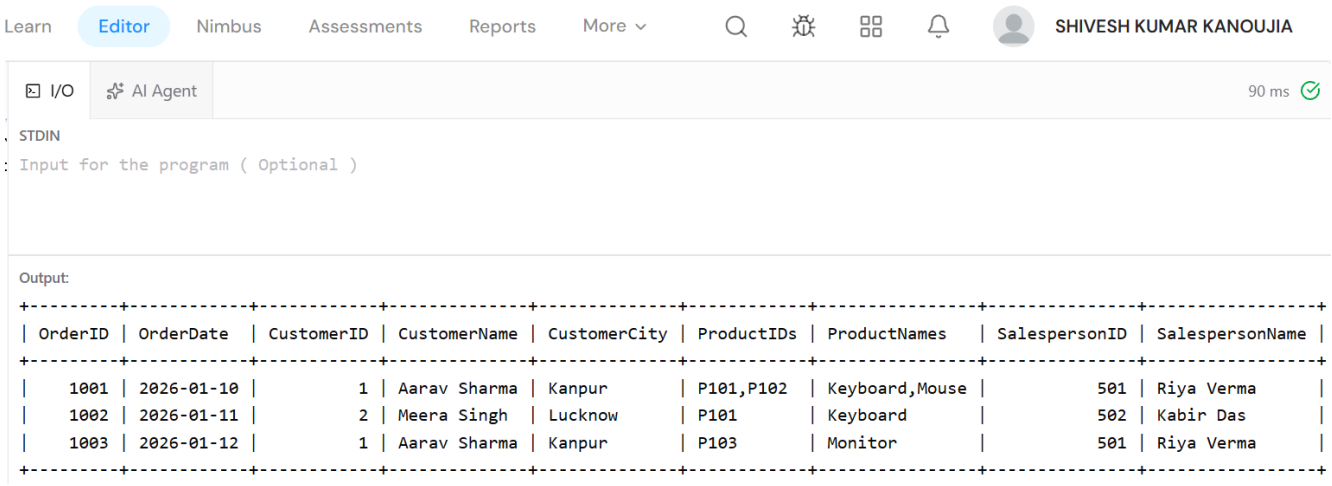
The initial Sales table stores order, customer, product, and salesperson details together. The ProductIDs and ProductNames columns may contain multiple values in one row, illustrating UNF.

```
CREATE TABLE Sales_UNF (
    OrderID INT PRIMARY KEY,
    OrderDate DATE NOT NULL,
    CustomerID INT NOT NULL,
    CustomerName VARCHAR(100) NOT NULL,
    CustomerCity VARCHAR(60),
    ProductIDs VARCHAR(100) NOT NULL,
    ProductNames VARCHAR(200) NOT NULL,
    SalespersonID INT NOT NULL,
    SalespersonName VARCHAR(100) NOT NULL
);

INSERT INTO Sales_UNF VALUES
(1001, '2026-01-10', 1, 'Aarav Sharma', 'Kanpur',
 'P101,P102', 'Keyboard,Mouse', 501, 'Riya Verma'),
(1002, '2026-01-11', 2, 'Meera Singh', 'Lucknow',
 'P101', 'Keyboard', 502, 'Kabir Das'),
(1003, '2026-01-12', 1, 'Aarav Sharma', 'Kanpur',
 'P103', 'Monitor', 501, 'Riya Verma');

SELECT * FROM Sales_UNF;
```

Output



2. IDENTIFY FUNCTIONAL DEPENDENCIES

For this sample dataset, assume each OrderID identifies one order, each CustomerID identifies one customer, each ProductID identifies one product, and each SalespersonID identifies one salesperson.

```
OrderID -> OrderDate, CustomerID, SalespersonID
CustomerID -> CustomerName, CustomerCity
ProductID -> ProductName
SalespersonID -> SalespersonName
(OrderID, ProductID) -> Quantity, UnitPrice

-- Candidate key for an order line: (OrderID, ProductID)
-- In the original UNF table, OrderID identifies the stored order record,
-- but repeating groups must be removed before line-level keys can be used.
```

Output

-- Functional Dependencies:

-- OrderID -> OrderDate, CustomerID, SalespersonID

-- CustomerID -> CustomerName, CustomerCity

-- ProductID -> ProductName

-- SalespersonID -> SalespersonName

-- (OrderID, ProductID) -> Quantity, UnitPrice

3. CONVERT UNF TO FIRST NORMAL FORM (1NF)

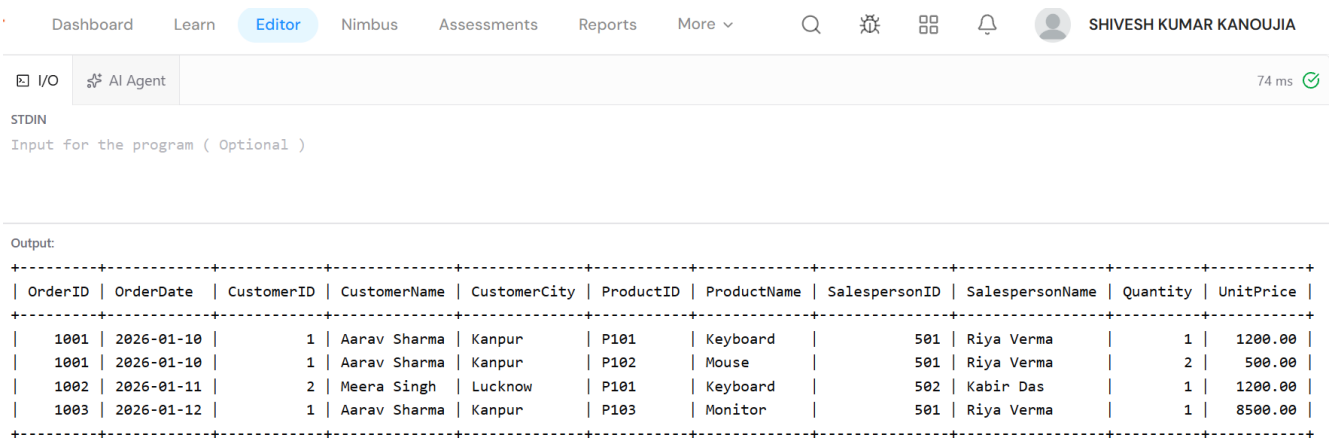
1NF requires each attribute to contain a single atomic value. Create one row per product in an order. Quantity and UnitPrice are included at the order-line level.

```
CREATE TABLE Sales_1NF (
    OrderID INT NOT NULL,
    OrderDate DATE NOT NULL,
    CustomerID INT NOT NULL,
    CustomerName VARCHAR(100) NOT NULL,
    CustomerCity VARCHAR(60),
    ProductID VARCHAR(10) NOT NULL,
    ProductName VARCHAR(100) NOT NULL,
    SalespersonID INT NOT NULL,
    SalespersonName VARCHAR(100) NOT NULL,
    Quantity INT NOT NULL DEFAULT 1,
    UnitPrice DECIMAL(10,2) NOT NULL DEFAULT 0.00,
    PRIMARY KEY (OrderID, ProductID)
);

INSERT INTO Sales_1NF VALUES
(1001, '2026-01-10', 1, 'Aarav Sharma', 'Kanpur', 'P101', 'Keyboard', 501, 'Riya
Verma', 1, 1200.00),
(1001, '2026-01-10', 1, 'Aarav Sharma', 'Kanpur', 'P102', 'Mouse',    501, 'Riya
Verma', 2, 500.00),
(1002, '2026-01-11', 2, 'Meera Singh',  'Lucknow','P101', 'Keyboard', 502,
'Kabir Das',  1, 1200.00),
(1003, '2026-01-12', 1, 'Aarav Sharma', 'Kanpur', 'P103', 'Monitor',  501, 'Riya
Verma', 1, 8500.00);

SELECT * FROM Sales_1NF ORDER BY OrderID, ProductID;
```

Output



Output:

OrderID	OrderDate	CustomerID	CustomerName	CustomerCity	ProductID	ProductName	SalespersonID	SalespersonName	Quantity	UnitPrice
1001	2026-01-10	1	Aarav Sharma	Kanpur	P101	Keyboard	501	Riya Verma	1	1200.00
1001	2026-01-10	1	Aarav Sharma	Kanpur	P102	Mouse	501	Riya Verma	2	500.00
1002	2026-01-11	2	Meera Singh	Lucknow	P101	Keyboard	502	Kabir Das	1	1200.00
1003	2026-01-12	1	Aarav Sharma	Kanpur	P103	Monitor	501	Riya Verma	1	8500.00

4. CONVERT 1NF TO SECOND NORMAL FORM (2NF)

2NF removes partial dependencies on part of a composite key. In Sales_1NF, order details depend on OrderID, while product details depend on ProductID. Separate orders, products, and order lines.

```
CREATE TABLE Orders_2NF (
    OrderID INT PRIMARY KEY,
    OrderDate DATE NOT NULL,
    CustomerID INT NOT NULL,
    CustomerName VARCHAR(100) NOT NULL,
    CustomerCity VARCHAR(60),
    SalespersonID INT NOT NULL,
    SalespersonName VARCHAR(100) NOT NULL
);

CREATE TABLE Products_2NF (
    ProductID VARCHAR(10) PRIMARY KEY,
    ProductName VARCHAR(100) NOT NULL
);

CREATE TABLE OrderItems_2NF (
    OrderID INT NOT NULL,
    ProductID VARCHAR(10) NOT NULL,
    Quantity INT NOT NULL,
    UnitPrice DECIMAL(10,2) NOT NULL,
    PRIMARY KEY (OrderID, ProductID),
    FOREIGN KEY (OrderID) REFERENCES Orders_2NF(OrderID),
    FOREIGN KEY (ProductID) REFERENCES Products_2NF(ProductID)
);

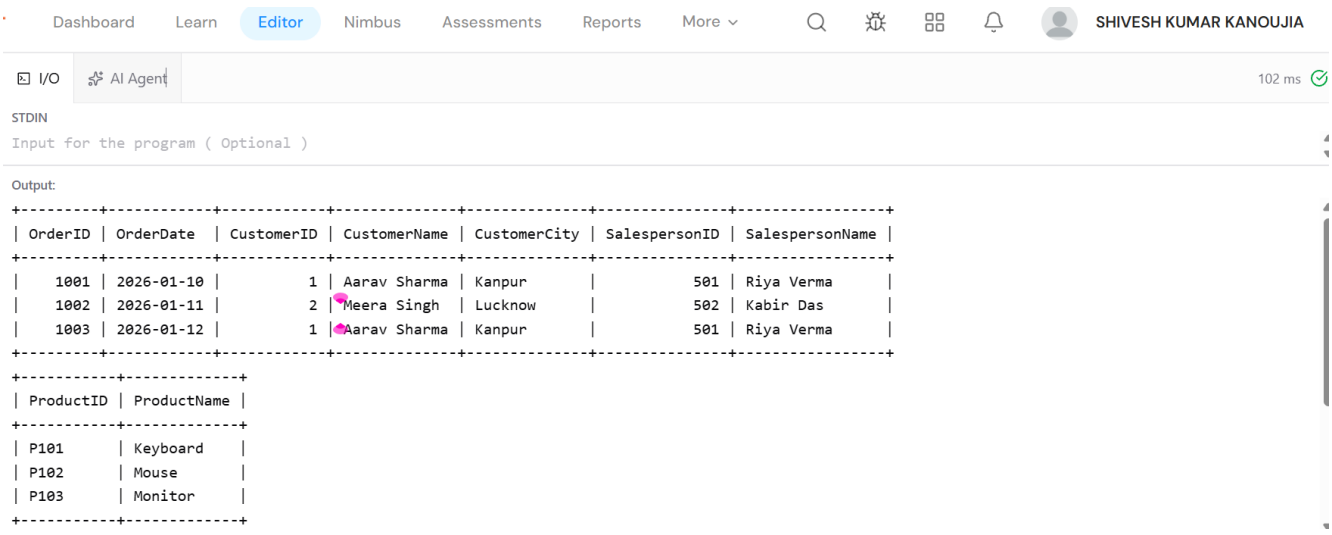
INSERT INTO Orders_2NF VALUES
(1001, '2026-01-10', 1, 'Aarav Sharma', 'Kanpur', 501, 'Riya Verma'),
(1002, '2026-01-11', 2, 'Meera Singh', 'Lucknow', 502, 'Kabir Das'),
(1003, '2026-01-12', 1, 'Aarav Sharma', 'Kanpur', 501, 'Riya Verma');

INSERT INTO Products_2NF VALUES
('P101', 'Keyboard'), ('P102', 'Mouse'), ('P103', 'Monitor');

INSERT INTO OrderItems_2NF VALUES
(1001, 'P101', 1, 1200.00), (1001, 'P102', 2, 500.00),
(1002, 'P101', 1, 1200.00), (1003, 'P103', 1, 8500.00);

SELECT * FROM Orders_2NF;
SELECT * FROM Products_2NF;
SELECT * FROM OrderItems_2NF;
```

Output



OrderID	ProductID	Quantity	UnitPrice
1001	P101	1	1200.00
1001	P102	2	500.00
1002	P101	1	1200.00
1003	P103	1	8500.00

5. CONVERT 2NF TO THIRD NORMAL FORM (3NF)

3NF removes transitive dependencies. CustomerID determines customer details, and SalespersonID determines salesperson details. Store these details in separate tables.

```
CREATE TABLE Customers (
    CustomerID INT PRIMARY KEY,
    CustomerName VARCHAR(100) NOT NULL,
    CustomerCity VARCHAR(60)
);

CREATE TABLE Salespersons (
    SalespersonID INT PRIMARY KEY,
    SalespersonName VARCHAR(100) NOT NULL
);

CREATE TABLE Orders_3NF (
    OrderID INT PRIMARY KEY,
    OrderDate DATE NOT NULL,
```

```
    CustomerID INT NOT NULL,
    SalespersonID INT NOT NULL,
    FOREIGN KEY (CustomerID) REFERENCES Customers(CustomerID),
    FOREIGN KEY (SalespersonID) REFERENCES Salespersons(SalespersonID)
);

INSERT INTO Customers VALUES
(1, 'Aarav Sharma', 'Kanpur'),
(2, 'Meera Singh', 'Lucknow');

INSERT INTO Salespersons VALUES
(501, 'Riya Verma'),
(502, 'Kabir Das');

INSERT INTO Orders_3NF VALUES
(1001, '2026-01-10', 1, 501),
(1002, '2026-01-11', 2, 502),
(1003, '2026-01-12', 1, 501);

SELECT o.OrderID, o.OrderDate, c.CustomerName, c.CustomerCity,
       s.SalespersonName
FROM Orders_3NF o
JOIN Customers c ON o.CustomerID = c.CustomerID
JOIN Salespersons s ON o.SalespersonID = s.SalespersonID
ORDER BY o.OrderID;
```

Output



6. VERIFY BCNF

BCNF requires every determinant of a non-trivial functional dependency to be a superkey. In the 3NF design above, CustomerID, SalespersonID, ProductID, and OrderID are keys of their respective tables. The order-line table uses the composite key (OrderID, ProductID).

```
-- Check primary keys and foreign-key relationships
SHOW CREATE TABLE Customers;
SHOW CREATE TABLE Salespersons;
SHOW CREATE TABLE Orders_3NF;
SHOW CREATE TABLE Products_2NF;
SHOW CREATE TABLE OrderItems_2NF;

-- Check that each order line is uniquely identified
SELECT OrderID, ProductID, COUNT(*) AS row_count
FROM OrderItems_2NF
GROUP BY OrderID, ProductID
HAVING COUNT(*) > 1;
```

Expected result: the duplicate-check query returns no rows. Under the stated functional dependencies, the decomposed tables satisfy BCNF.

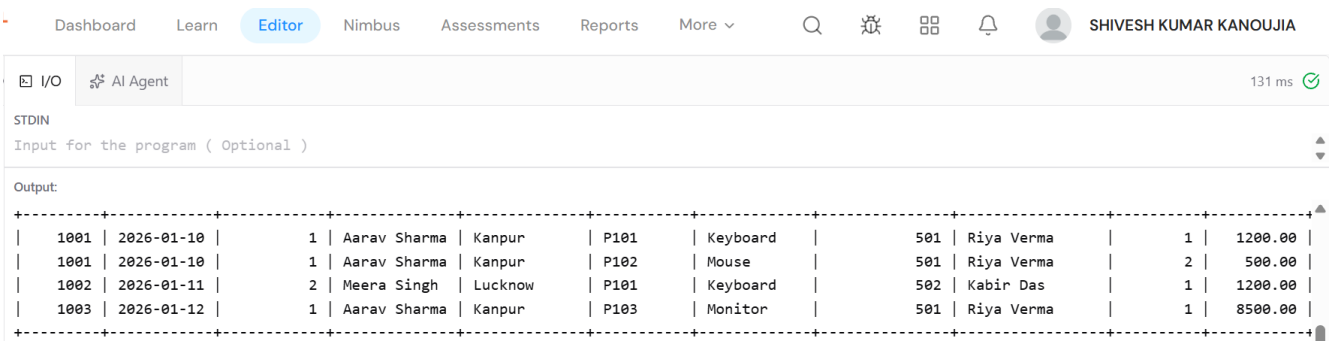
7. VERIFY LOSSLESS JOIN

A decomposition is lossless if joining the decomposed relations reconstructs the original atomic order-line information without adding spurious rows.

```
SELECT o.OrderID, o.OrderDate,
       c.CustomerID, c.CustomerName, c.CustomerCity,
       p.ProductID, p.ProductName,
       s.SalespersonID, s.SalespersonName,
       i.Quantity, i.UnitPrice
FROM Orders_3NF o
JOIN Customers c ON o.CustomerID = c.CustomerID
JOIN Salespersons s ON o.SalespersonID = s.SalespersonID
JOIN OrderItems_2NF i ON o.OrderID = i.OrderID
JOIN Products_2NF p ON i.ProductID = p.ProductID
ORDER BY o.OrderID, p.ProductID;

-- Compare the reconstructed row count with the number of order lines
SELECT COUNT(*) AS reconstructed_rows
FROM Orders_3NF o
JOIN OrderItems_2NF i ON o.OrderID = i.OrderID
JOIN Products_2NF p ON i.ProductID = p.ProductID;
```

Expected result: four reconstructed order-line rows, matching the four rows in OrderItems_2NF.



8. DISCUSS DEPENDENCY PRESERVATION

The decomposition preserves the listed dependencies because customer details remain in Customers, salesperson details remain in Salespersons, order details remain in Orders_3NF, product names remain in Products_2NF, and quantity/unit price remain in OrderItems_2NF. Each dependency can be checked within its own relation without needing to join all tables.

```
-- Customer dependency: CustomerID -> CustomerName, CustomerCity
SELECT CustomerID, COUNT(*) AS rows_per_customer
FROM Customers
GROUP BY CustomerID
HAVING COUNT(*) > 1;

-- Product dependency: ProductID -> ProductName
SELECT ProductID, COUNT(*) AS rows_per_product
FROM Products_2NF
GROUP BY ProductID
HAVING COUNT(*) > 1;

-- Salesperson dependency: SalespersonID -> SalespersonName
SELECT SalespersonID, COUNT(*) AS rows_per_salesperson
FROM Salespersons
GROUP BY SalespersonID
HAVING COUNT(*) > 1;
```

Expected result: all three validation queries return no rows.

RESULT

The denormalized Sales data was analyzed for functional dependencies and decomposed step-by-step into 1NF, 2NF, 3NF, and BCNF. SQL queries were used to inspect the normalized relations and verify lossless reconstruction and dependency preservation.

CONCLUSION

Normalization reduces repeated data and helps prevent insertion, update, and deletion anomalies. The final schema separates customers, salespersons, orders, products, and order items while retaining relationships through primary and foreign keys.